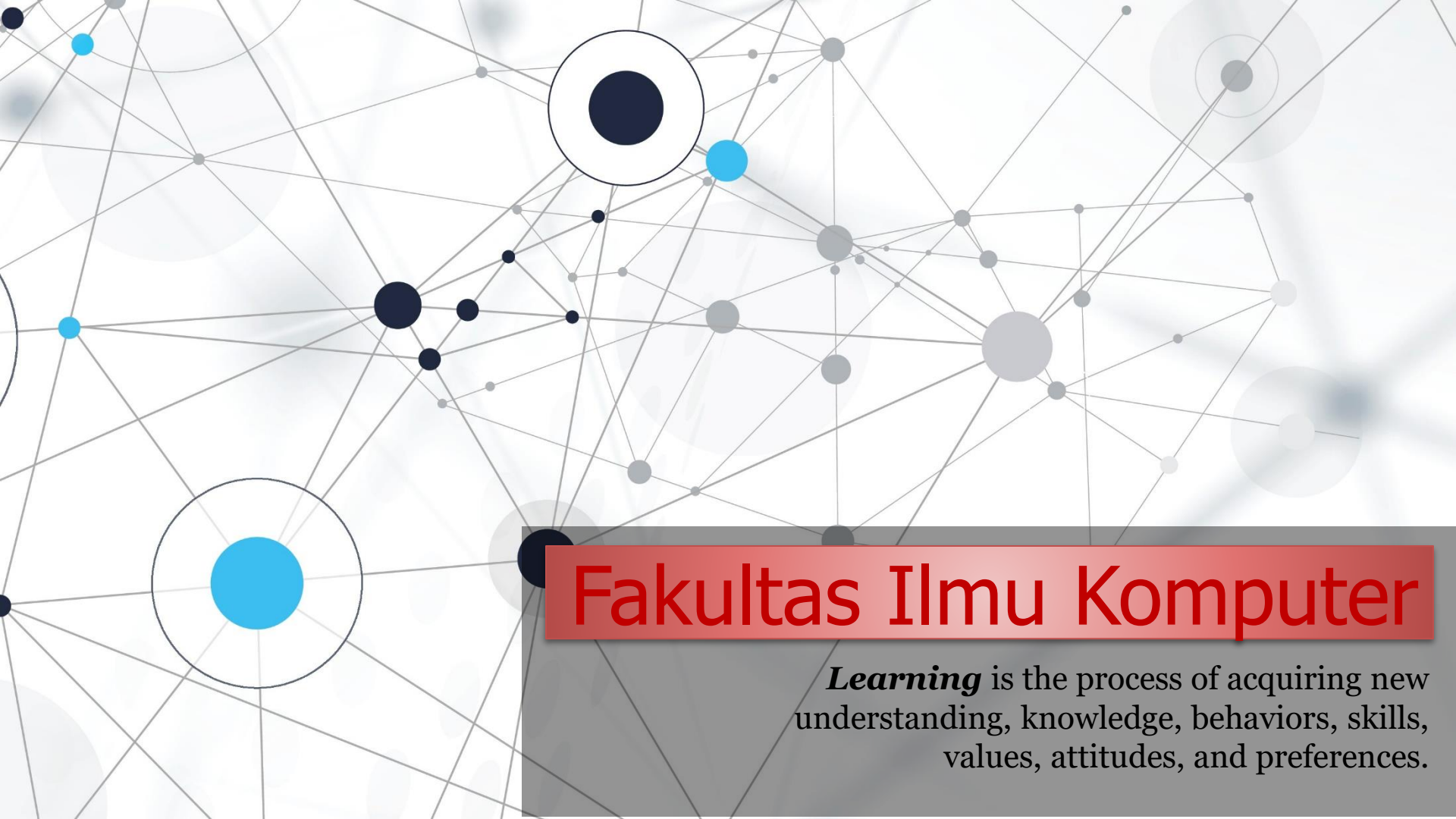




Fakultas Ilmu Komputer

Learning is the process of acquiring new understanding, knowledge, behaviors, skills, values, attitudes, and preferences.

OOP Quiz #2

Question #1

```
# parent class
2 class MakhlukHidup:
3     def __init__(self, nama):
4         self.nama = nama
5
6     def ciri(self):
7         print("Ini adalah makhluk hidup.")
8
9 # child class
10 class Hewan(MakhlukHidup):
11     def __init__(self, nama, habitat):
12         super().__init__(nama)
13         self.habitat = habitat
14
15     def ciri(self):
16         #super().ciri() # call method parent class
17         print(f"{self.nama} adalah hewan yang hidup di {self.habitat}.")
18
19 # create object
20 singa = Hewan("Singa", "hutan", "putih")
21
22 # call method
23 singa.ciri()
```

Perhatikan contoh source code sederhana Single Inheritance dengan Python menggunakan class Hewan. Pilihlah output yang tepat dari source pada gambar di atas.

- A. Singa adalah hewan yang hidup di hutan.
- B. Hutan adalah hewan yang hidup di singa.
- C. Singa bukan hewan yang hidup di hutan.
- D. NameError: name 'hewan' is not defined.
- E. TypeError: __init__() takes 3 positional arguments but 4 were given.

Question #2

```
1 # parent class 1
2 class Pegawai:
3     def __init__(self, nama, id_pegawai):
4         self.nama = nama
5         self.id_pegawai = id_pegawai
6
7     def info_pegawai(self):
8         print(f"Pegawai: {self.nama}, ID Pegawai: {self.id_pegawai}")
9
10 # parent class 2
11 class Dosen:
12     def __init__(self, nama, nidn):
13         self.nama = nama
14         self.nidn = nidn
15
16     def info_dosen(self):
17         print(f"Dosen: {self.nama}, NIDN: {self.nidn}")
18
19 # child class
20 class pengajar(Pegawai, Dosen):
21     def __init__(self, nama, id_pegawai, nidn, mata_kuliah):
22         Pegawai.__init__(self, nama, id_pegawai) # init class Pegawai
23         Dosen.__init__(self, nama, nidn) # init class Dosen
24         self.mata_kuliah = mata_kuliah
25
26     def info_pengajar(self):
27         print(f"INFO LENGKAP PENGAJAR CAMPUS:")
28         print(f"Pengajar: {self.nama}")
29         print(f"ID Pegawai: {self.id_pegawai}")
30         print(f"NIDN: {self.nidn}")
31         print(f"Mata Kuliah: {self.mata_kuliah}")
32
33
34 # create object and call methods
35 pengajar1 = Pengajar("Dr. John Doe", "123", "456789", "Pemrograman Python")
36 pengajar1.info_pegawai()
37 pengajar1.info_dosen()
38 pengajar1.info_pengajar()
```

Berikut ini adalah contoh sederhana Multiple Inheritance dengan Python menggunakan class Pegawai, Dosen, dan Pengajar. Pilihlah output yang tepat dari gambar di samping ini:

- A. **NameError: name 'Pengajar' is not defined.**
- B. **NameError: name 'Dosen' is not defined.**
- C. **TypeError: object.__init__() takes exactly one argument (the instance to initialize).**
- D. **TypeError: __init__() missing 1 required positional argument: 'mata_kuliah'**
- E. **SyntaxError: invalid syntax in class Pengajar(Pegawai, Dosen);**

Question #3

```
1 # parent class
2 class Bentuk:
3     def deskripsi(self):
4         return "Ini adalah bentuk geometri."
5
6 # child class 1
7 class Persegi(Bentuk):
8     def deskripsi(self):
9         return "Ini adalah persegi."
10
11 # child class 2
12 class Lingkaran(Bentuk):
13     def deskripsi(self):
14         return "Ini adalah lingkaran."
15
16 # child class 3
17 class Segitiga(Bentuk):
18     def deskripsi(self):
19         return "Ini adalah segitiga."
20
21 # create objects
22 bentuk = Bentuk()
23 persegi = Persegi()
24 lingkaran = Lingkaran()
25 segitiga = Segitiga()
26
27 # call methods
28 print(segitiga.deskripsi())
```

Berikut ini adalah contoh sederhana untuk Method Overriding menggunakan Python. Output yang akan ditampilkan jika program di samping akan di execute adalah?

- A. Ini adalah bentuk geometri.
- B. Ini adalah persegi.
- C. Ini adalah lingkaran.
- D. Ini adalah segitiga.
- E. **TypeError: deskripsi() takes 1 positional argument but 2 were given.**

Question #4

```
1  # class
2  class Calculator:
3  def __init__(self):
4      pass
5
6  def add(self, a, b=0):
7      return a + b
8
9      def subtract(self, a, b=0):
10         return a - b
11
12     def multiply(self, a, b=1):
13         return a * b
14
15     def divide(self, a, b=1):
16         if b == 0:
17             return "Error: Pembagian dengan nol!"
18         return a / b
19
20     def power(self, a, b=2):
21         return a ** b
22
23 # create object
24 calc = Calculator()
25
26 # Panggil metode divide()
27 result1 = calc.divide(5, 2)
28 print("Hasil Pembagian:", result1)
```

Dalam Python, tidak ada dukungan untuk method overloading. Namun, kita dapat menggunakan parameter default untuk mencapai hasil yang serupa. Perhatikan code di samping dan pilihlah output yang sesuai?

A. Hasil Pembagian: 2.5

B. NameError: name 'Calculator' is not defined.

C. TypeError: Calculator() takes no arguments.

D. Hasil Pembagian: Error: Pembagian dengan nol!

E. IndentationError: expected an indented block.

Question #5

```
1 # parent class
2 class Manusia:
3     def __init__(self, nama):
4         self.nama = nama
5
6     def get_nama(self):
7         return self.nama
8
9     def set_nama(self, nama):
10        self.nama = nama
11
12 # child class
13 class Mahasiswa(Manusia):
14     def __init__(self, self, nama, jurusan):
15         super().__init__(nama) # init parent class
16         self.jurusan = jurusan
17
18     def get_jurusan(self):
19         return self.jurusan
20
21     def set_jurusan(self, jurusan):
22         self.jurusan = jurusan
23
24 # Contoh penggunaan
25 mahasiswa = Mahasiswa("Semmy Taju", "Informatika")
26
27 # Mendapatkan informasi
28 print("INFORMASI AWAL YANG DINPUTKAN:")
29 print("Nama:", mahasiswa.get_nama())
30 print("Jurusan:", mahasiswa.get_jurusan())
31
32 # Mengubah informasi
33 mahasiswa.set_nama("Albert Toda")
34 mahasiswa.set_jurusan("Sistem Informasi")
35
36 # Mengkonfirmasi perubahan
37 print("\nSETELAH PERUBAHAN INFORMASI:")
38 print("Nama:", mahasiswa.get_nama())
39 print("Jurusan:", mahasiswa.get_jurusan())
```

Berikut ini adalah contoh dengan menggunakan method setters dan getters dalam pewarisan. Temukan error yang tepat pada gambar di samping?

- A. **NameError: name 'manusia' is not defined.**
- B. **NameError: name 'Mahasiswa' is not defined.**
- C. **SyntaxError: duplicate argument 'self' in function definition.**
- D. **TypeError: __init__() takes 3 positional arguments but 4 were given.**
- E. **TypeError: get_nama() takes 1 positional argument but 2 were given.**

Question #6

```
# class independent #1
1 class Mobil:
2     def __init__(self, merk, warna):
3         self.merk = merk
4         self.warna = warna
5
6     def info_mobil(self):
7         return f"Mobil {self.warna} merk {self.merk}"
8
9
10 # class independent #2
11 class Pemilik:
12     def __init__(self, nama, mobil):
13         self.nama = nama
14         self.mobil = mobil
15
16     def info_pemilik(self):
17         print(f"Pemilik: {self.nama}")
18         print(f"Jenis mobil: {self.mobil.info_mobil()}")
19
20 # create objects from each class
21 mobil1 = Mobil("Sigra", "Hitam")
22 pemilik1 = Pemilik("Semmy Taju", mobil1)
23
24 # call method
25 pemilik1.info_pemilik()
```

Dengan menggunakan konsep Composition, menunjukkan bahwa satu class dapat memiliki ketergantungan pada kelas lain melalui objeknya. Selidiki kesalahan code di samping ini?

- A. **IndentationError: expected an indented block.**
- B. **TypeError: info_pemilik() takes 1 positional argument but 2 were given.**
- C. **TypeError: __init__() takes 3 positional arguments but 4 were given.**
- D. **Pemilik: Semmy Taju
Jenis mobil: Mobil Hitam merk Sigra**
- E. **NameError: name 'pemilik' is not defined.**

Question #7

```
1 # create class
2 class Calculator:
3     def add(self, x, y):
4         try:
5             result = x + y
6             return result
7         except Exception as e:
8             return f"Error: {e}"
9
10    def subtract(self, x, y):
11        try:
12            result = x - y
13            return result
14        except Exception as e:
15            return f"Error: {e}"
16
17    def multiply(self, x, y):
18        try:
19            result = x * y
20            return result
21        except Exception as e:
22            return f"Error: {e}"
23
24    def divide(self, x, y):
25        try:
26            result = x / y
27            return result
28        except ZeroDivisionError:
29            return "Error: Pembagian dengan angka nol tidak diizinkan."
30        except Exception as e:
31            return f"Error: {e}"
32
33 # create object
34 calculator = Calculator()
35
36 # Pembagian
37 print("Pembagian:", calculator.divide(10, 2))
```

Source code di samping adalah contoh implementasi kelas Calculator beserta penanganan error pada setiap fungsi. Pilihlah output yang tepat pada saat code di samping di run?

- A. **Pembagian: Error: Pembagian dengan angka nol tidak diizinkan.**
- B. **SyntaxError: invalid syntax in axcept ZeroDivisionError:**
- C. **NameError: name 'Calcalator' is not defined.**
- D. **Pembagian: 5.0**
- E. **IndentationError: expected an indented block.**

Question #8

Source code di bawah ini adalah contoh penanganan error di Python menggunakan *try*, *except* dan *finally* untuk `ArithmeticError`. Pilihlah code yang benar tanpa kesalahan penulisan dan bisa di execute?

(A)

```
1 try:
2     hasil = 10 / 0 # ArithmeticError
3 except AritmeticError as e:
4     print("Error type:", e)
5 finally:
6     print("Done")
```

(D)

```
1 try:
2     hasil = 10 / 0 # ArithmeticError
3 except ArithmeticError as e:
4     print("Error type:", e)
5 finally:
6     print("Done")
```

(B)

```
1 try:
2     hasil = 10 / 0 # ArithmeticError
3 except ArithmeticError as e:
4     print("Error type:", e)
5 finelly:
6     print("Done")
```

(E)

```
1 try:
2     hasil = 10 / 0 # ArithmeticError
3 except ArithmeticError as e:
4     print("Error type:", a)
5 finally:
6     print("Done")
```

(C)

```
1 try:
2     hasil = 10 / 0 # ArithmeticError
3 except ArithmeticError as e:
4     print("Error type:", e)
5 finally:
6     print("Done")
7
```

Question #9

Code di samping menunjukkan contoh penggunaan regex di Python untuk mengekstrak pola dengan format yang diberikan. Pilihlah output yang sesuai???

```
1 # import package
2 import re
3
4 # input text
5 input_string = "semmy@unklab.com 12/09/2024 3PM 5 orang siang"
6
7 # regex pattern
8 pattern = r'(\d{2})/(\d{2})/(\d{4})'
9
10 # match pola regex dengan string input
11 matches = re.search(pattern, input_string)
12
13 # extract output pattern
14 if matches:
15     output = matches.group(0)
16     print(output)
17 else:
18     print("Format tidak ditemukan.")
19
```

A. semmy@unklab.com

B. 12/09/2024

C. 3PM

D. NameError: name 're' is not defined.

E. IndexError: no such group.

Question #10

```
1 # import module
2 import re
3
4 # create class
5 class GradingSystem:
6     def __init__(self):
7         self.grade_regex = r'^[0-9]+(\.[0-9]+)?$'
8
9     def grade(self, score):
10        if re.match(self.grade_regex, score):
11            score = float(score)
12            if score >= 90:
13                return 'A'
14            elif score >= 80:
15                return 'B'
16            elif score >= 70:
17                return 'C'
18            elif score >= 60:
19                return 'D'
20            else:
21                return 'E'
22        else:
23            return 'Invalid input'
24
25 # create object
26 demo = GradingSystem()
27
28 # input student score
29 scores = ['95', '85.5', '75', '65.7', 'abc']
30
31 # view grade
32 for score in scores:
33     print(f"Score: {score}, Grade: {demo.grade(score)}")
```

Pilihlah output dari program di samping jika di execute?

- A. **AttributeError: module 're' has no attribute 'metch'.**
- B. **AttributeError: 'GradingSystem' object has no attribute 'grade_regex'.**
- C. **error: internal: unsupported template operator MAX_REPEAT.**
- D. **Score: 95, Grade: A
Score: 85.5, Grade: B
Score: 75, Grade: C
Score: 65.7, Grade: D
Score: abc, Grade: Invalid input**
- E. **Score: 95, Grade: A
Score: 85.5, Grade: B
Score: 75, Grade: C
Score: 65.7, Grade: D
Score: abc, Grade: Valid input**

**trust
yourself.
you know
more than you
think you do.**

(dr. spock)

END PRESENTATION

Thank you for your attention

Instructor: S – W – T

THANK YOU

